JOB PORTAL SYTEM

A PROJECT REPORT

Submitted by

Sahil khatkar-24BCS12472

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

Chandigarh University

INTRODUCTION

The Job Portal System is a full-stack web application designed to connect job seekers with available job opportunities. The system is developed using React.js for the frontend and Node.js with Express.js for the backend.

In today's digital world, job searching has shifted from traditional methods to online platforms. This project aims to simulate a simplified version of such platforms, allowing users to browse jobs, view details, and apply for positions.

The application follows the concept of a Single Page Application (SPA), where navigation between pages happens without reloading the entire website. This improves user experience and performance.

The backend provides RESTful APIs that handle job data and application submissions. Instead of using a database, job data is stored in a JavaScript array, making the project lightweight and beginner-friendly.

OBJECTIVES

The main objectives of this project are:

- To develop a **basic full-stack web application**
- To understand **client-server architecture**
- To implement **REST APIs using Express.js**
- To build a dynamic user interface using **React**
- To implement **search and filtering functionality**
- To simulate real-world job portal behavior
- To create a project that is **easy to explain in viva**

FUNCTIONAL REQUIREMENTS :

User Functional Requirements

1. View Job Listings

- The system should display all available jobs.
- Each job should be shown as a **card** containing:
  - Job title
  - Company name
  - Location
  - Category

2. Search Jobs

- User should be able to search jobs using keywords.
- Search should work on:
  - Job title
  - Company name
- The system filters results dynamically.

3. Filter Jobs by Category

- Users should be able to filter jobs based on categories such as:
    - IT
    - Marketing
    - Finance
- Only matching jobs should be displayed.

4. View Job Details

- User should be able to click on a job and view:
    - Full description
    - Requirements
    - Salary
    - Location

5. Apply for Job

- User should be able to fill an application form.
- Form fields:
    - Name
    - Email
    - Phone
    - Resume (text input)

6. Application Submission

- After submission:
    - System should validate input
    - Send data to backend
    - Show success message

System Functional Requirements

1. API to Fetch All Jobs

- Endpoint: GET /api/jobs
- Returns all job listings

2. API to Fetch Job by ID

- Endpoint: GET /api/jobs/:id
- Returns specific job details

3. API to Submit Application

- Endpoint: POST /api/apply
- Accepts form data
- Sends success response

4. Data Handling

- Job data stored in jobs.js
- Data is sent in JSON format

4. SYSTEM WORKFLOW

Step 1: User Access

- User opens the website
- Home page is displayed

Step 2: Navigation to Jobs Page

- User clicks “Explore Jobs”
- React Router loads Jobs page

Step 3: Fetch Jobs from Backend

- Frontend sends request:

GET /api/jobs

- Backend reads data from jobs.js
- Sends job list as response

Step 4: Display Jobs

- React stores jobs in state
- Displays them using JobCard component

Step 5: Search and Filter

- User types search keyword
- React filters job array using .filter()
- UI updates dynamically

Step 6: View Job Details

- User clicks “View Details”
- URL changes to /jobs/:id
- Frontend sends:

GET /api/jobs/:id

- Backend returns specific job

Step 7: Apply for Job

- User clicks “Apply Now”
- Form page opens
- User enters details

Step 8: Submit Application

- Frontend sends:

POST /api/apply

- Backend receives data
- Validates fields
- Logs application

Step 9: Confirmation

- Backend sends success message
- Frontend displays confirmation

5. DATA FLOW DIAGRAM (DFD)

Level 0

User → Frontend → Backend → Data Storage

Level 1

User Input → React UI → API Request → Express Server → jobs.js → Response → UI Update

6. SYSTEM ARCHITECTURE

The system follows **Client-Server Architecture**:

Frontend (Client)

- React.js
- Handles UI and user interaction

Backend (Server)

- Node.js + Express
- Handles API and data processing

Data Layer

- JavaScript array (jobs.js)

7. SYSTEM LAYOUT

Pages Structure

Home → Jobs → Job Details → Apply

UI Components

- Navbar
- Job Cards
- Search Bar
- Filter Dropdown
- Footer

8. LIMITATIONS

1. No Database

- Data is not stored permanently
- All jobs are hardcoded

2. No Authentication

- No login/signup system
- Anyone can apply

3. No Data Persistence

- Applications are not saved
- Data is lost on server restart

4. No Resume Upload

- Resume is only text input
- No file handling

5. Limited Scalability

- Cannot handle large number of users
- Not suitable for real-world deployment

6. No Admin Panel

- Cannot add/edit/delete jobs dynamically

9. FUTURE ENHANCEMENTS

- Add MongoDB database
- Implement login system
- Add admin dashboard
- Enable resume upload
- Improve UI/UX
- Deploy application online

10. CONCLUSION

The Job Portal System is a simple yet effective full-stack application that demonstrates how frontend and backend interact in a real-world system.

The project successfully implements:

- API communication
- Dynamic UI updates
- Form handling
- Search and filtering logic

Although it has limitations, it serves as a strong foundation for understanding full-stack development concepts.